Graphs

Course Name: Data Structures
Course Code: E1PY203B
Instructor Name: Dr. Kanchan Naithani

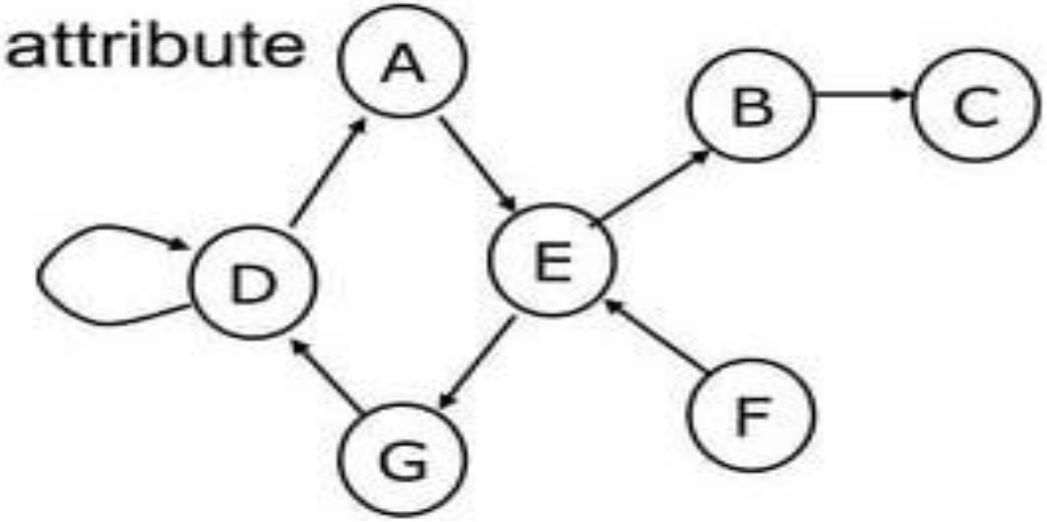
Unit-04

Basic Terminology and Representations – Graphs (Adjacency Matrices, Adjacency List) & Multi-Graphs - Directed Graphs - Adjacency Matrices - Graph Traversals (Depth First Search and Breadth First Search) - Spanning Trees - Minimum Cost Spanning Trees- Prims and Kruskal algorithm.

Graphs

A graph is a collection of nodes (or vertices, singular is vertex) and edges (or arcs)

- Each node contains an element
- Each edge connects two nodes together (or possibly the same node to itself) and may contain an edge attribute



A graph G is defined as follows:

$$G=(V,E)$$

$V(G)$: a finite, nonempty set of vertices

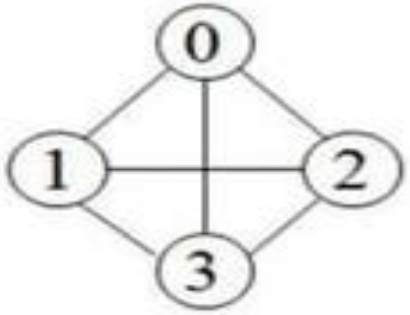
$E(G)$: a set of edges (pairs of vertices)

Types of graphs

- In discrete mathematics and graph theory, graphs can be classified into various types based on different characteristics. Here are some common types of graphs:
- **Directed Graph (Digraph):**
  - In a directed graph, each edge has a direction, indicating a one-way relationship between nodes.
  - The edges are represented by arrows, indicating the direction from one node to another.
- **Undirected Graph:**
  - In an undirected graph, edges have no direction, and the relationships between nodes are mutual.
  - Edges are typically represented as simple lines connecting nodes.
- **Weighted Graph:**
  - In a weighted graph, each edge is assigned a numerical weight or cost.
  - These weights may represent distances, costs, or any other relevant metric associated with the edges.
- **Unweighted Graph:**
  - In contrast to a weighted graph, an unweighted graph has no assigned weights on its edges.
- **Cyclic Graph:**
  - A cyclic graph contains at least one cycle (a closed path of edges) that starts and ends at the same node.
- **Acyclic Graph:**
  - An acyclic graph does not contain any cycles. It is a graph without closed loops.

Undirected graph

- An undirected graph is one in which the edges do not have a direction
- 'graph' denotes undirected graph.
- <u>Undirected graph:</u>
  - (v1, v2) in E is un-ordered.
  - (v1,v2) and (v2, v1) represent the same edge.



G_1

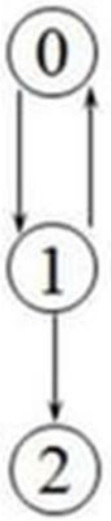
G1 = (4, 6)

V(G1) = { 0, 1, 2 , 3 }

E(G1) = { (0,1), (0,2), (0,3
(1,2), (1,3), (2,3) }

Directed graph

- Directed graph is one in which the edges have a direction
- Also called as 'digraph'
- <u>Directed graph:</u>
  - < v1, v2 > in E is ordered.
  - <V1,v2> and <v2, v1> represent the two different edges.



G_3

G3 = (3, 3)

V(G3) = { 0,1,2 }

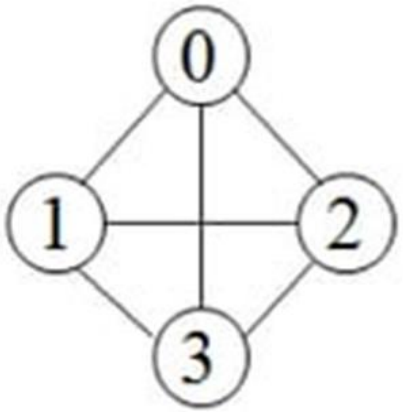
E(G3) = { <0,1> , <1,0> , <1,2>

Complete graph

A complete graph is a graph that has the maximum number of edges .

- for undirected graph with n vertices, the maximum number of edges is $n(n-1)/2$
- for directed graph with n vertices, the maximum number of edges is $n(n-1)$

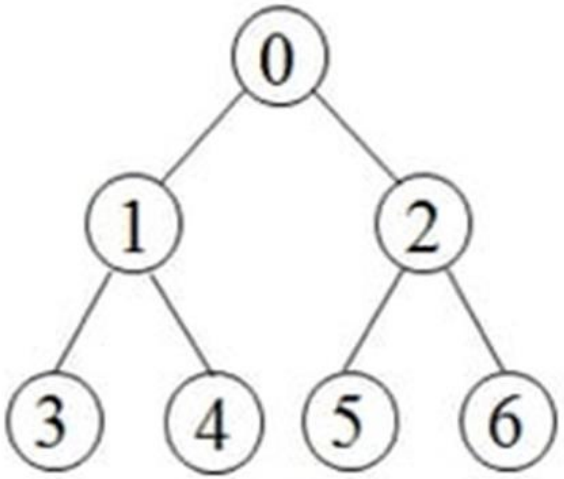
Complete graph



G_1

complete graph

No.of edges = n(n-1)/2
= 4*3/2
= 12/2
= 6



G_2

No.of edges = n(n-1)/2
= 7*6/2
= 42/2
= 21

incomplete graph

G_3

No.of edges = n(n-1
= 3*2
= 6

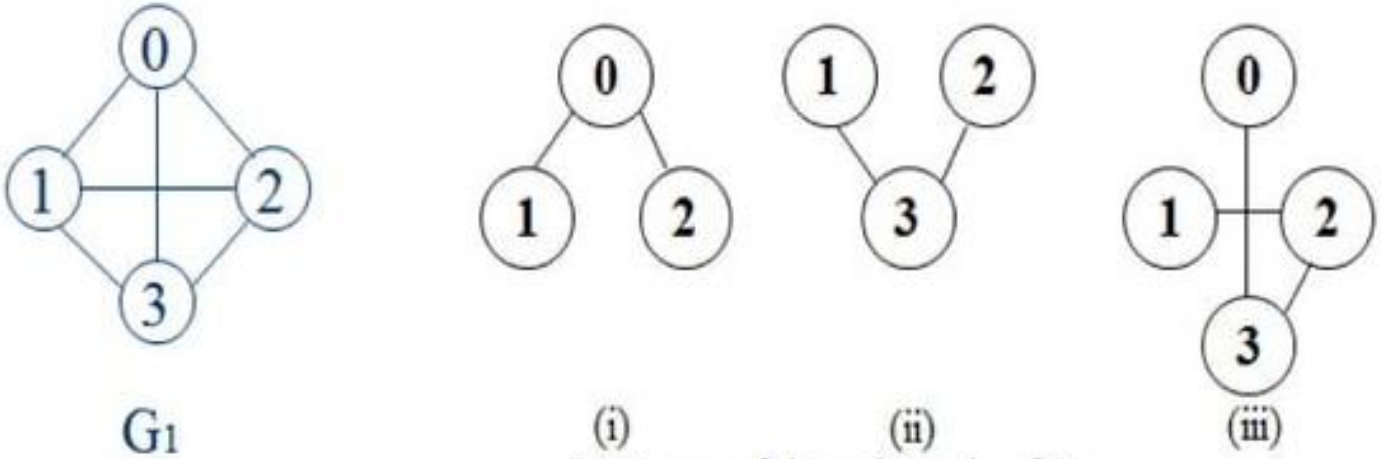
Adjacent and Incident

- If (v0, v1) is an edge in an undirected graph,
  - v0 and v1 are adjacent
  - The edge (v0, v1) is incident on vertices v0 and v1
- If <v0, v1> is an edge in a directed graph
  - v0 is adjacent to v1, and v1 is adjacent from v0
  - The edge <v0, v1> is incident on vertices v0 and v1

Sub- graph

A sub-graph of G is a graph G' such that

- V(G') is a subset of V(G)
- E(G') is a subset of E(G)



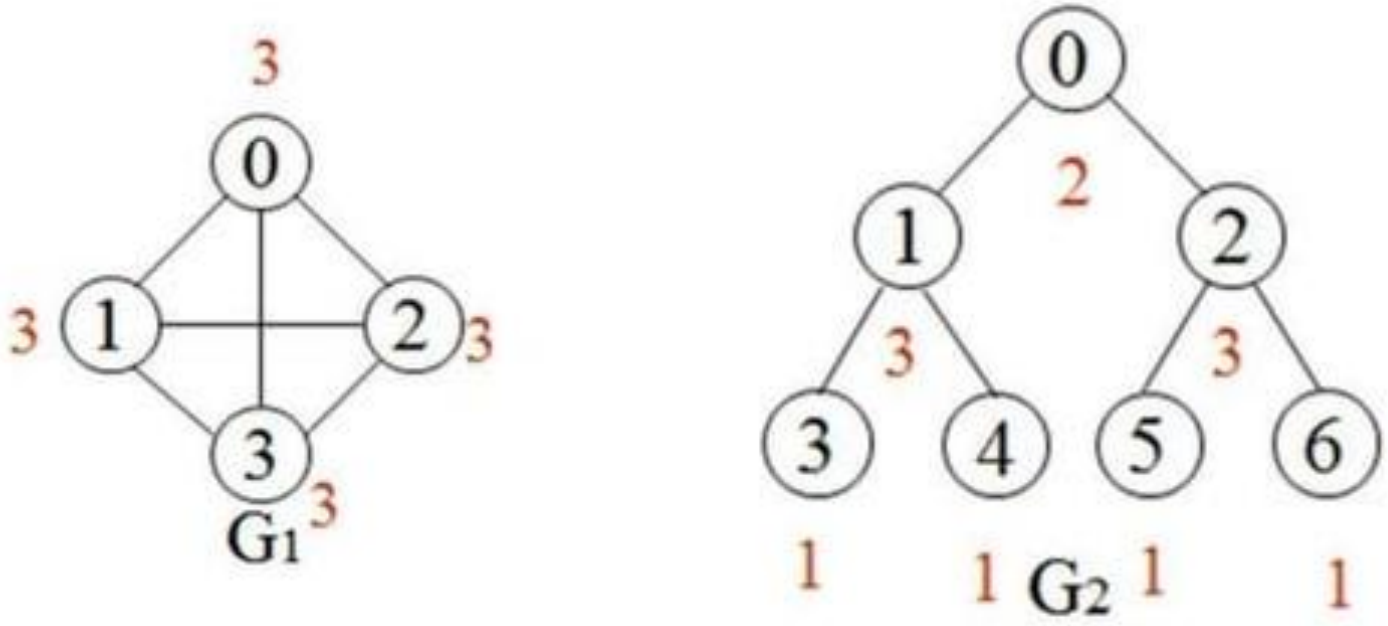
(a) Some of the subgraph of G_1

Degree

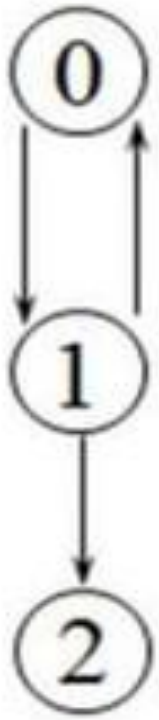
- The degree of a vertex is the number of edges incident to that vertex
- For directed graph,
  - the in-degree of a vertex *v* is the number of edges that have *v* as the head
  - the out-degree of a vertex *v* is the number of edges that have *v* as the tail
  - if *di* is the degree of a vertex *i* in a graph *G* with *n* vertices and *e* edges, the number of edges is

$$e = \left(\sum_{0}^{n-1} d_i\right) / 2$$

Degree - graph



Degree - digraph



in:1, out: 1

in: 1, out: 2

in: 1, out: 0

G_3

Graph Representation

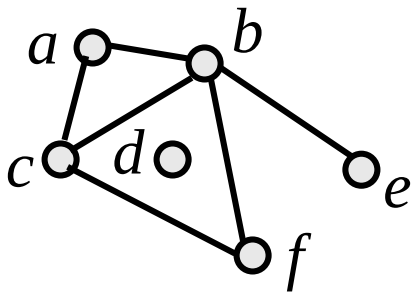
- Adjacency Matrix
- Adjacency Lists

Adjacency matrix

- Let G=(V,E) be a graph with n vertices.
- The adjacency matrix of G is a two-dimensional n by n array, say adj_mat
  - If the edge (vi, vj) is in E(G), adj_mat[i][j]=1
  - If there is no edge in E(G), adj_mat[i][j]=0
- The adjacency matrix for an undirected graph is symmetric
- the adjacency matrix for a digraph need not be symmetric

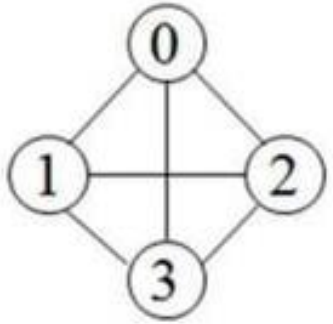
Adjacency Lists

- A table with 1 row per vertex, listing its adjacent vertices.



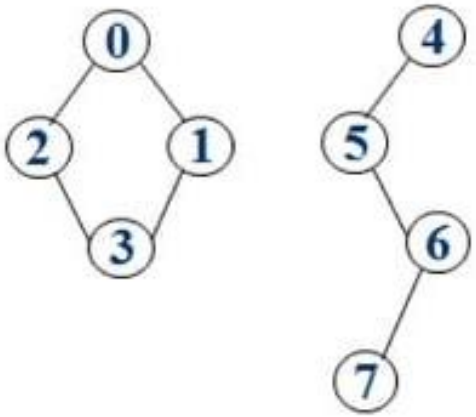
Vertex	*Adjacent Vertices*
a	*b, c*
b	*a, c, e, f*
c	*a, b, f*
d	
e	*b*
f	*c, b*

Adjacency matrix - graph



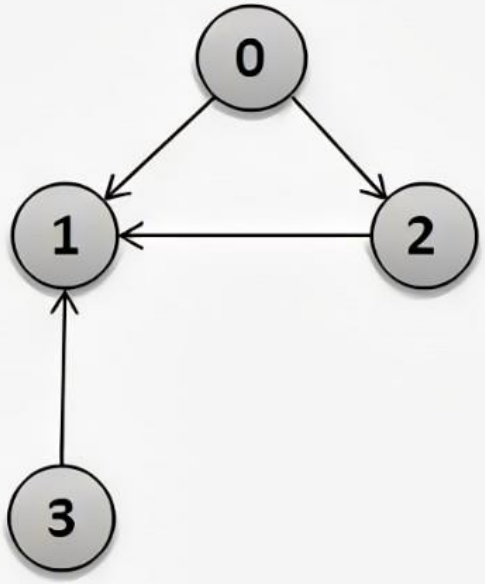
G_1

$$
\begin{array}{c|cccc}
 & 0 & 1 & 2 & 3 \\
\hline
0 & 0 & 1 & 1 & 1 \\
1 & 1 & 0 & 1 & 1 \\
2 & 1 & 1 & 0 & 1 \\
3 & 1 & 1 & 1 & 0
\end{array}
$$



$$
\begin{array}{c|cccccccc}
 & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\
\hline
0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\
1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
2 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
3 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\
4 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
5 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\
6 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\
7 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0
\end{array}
$$

Adjacency Matrix Representation of Directed Graph



	0	1	2	3
0	0	1	1	0
1	0	0	0	0
2	0	1	0	0
3	0	1	0	0

Adjacency matrix - digraph

G_3

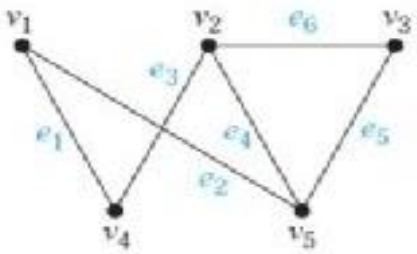
$$\begin{array}{c|ccc} & 0 & 1 & 2 \\ \hline 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 2 & 0 & 0 & 0 \end{array}$$

Incidence Matrix

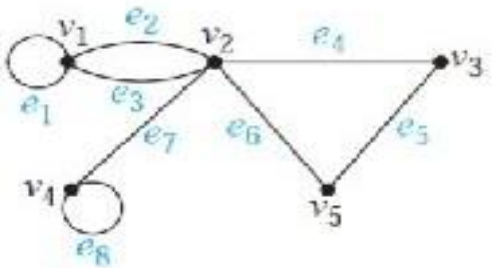
DEFINITION: Let $G = (V = \{v_1, \dots, v_n\}, E = \{e_1, \dots, e_m\})$ be <u>undirected</u>.
The **incidence matrix**关联矩阵 of G is an $n \times m$ matrix $B = (b_{ij})$, where

$$b_{ij} = \begin{cases} 1 & \text{if } e_j \text{ is incident with } v_i \\ 0 & \text{otherwise} \end{cases}$$

- e_j **incident with** v_i: v_i is an endpoint of e_j

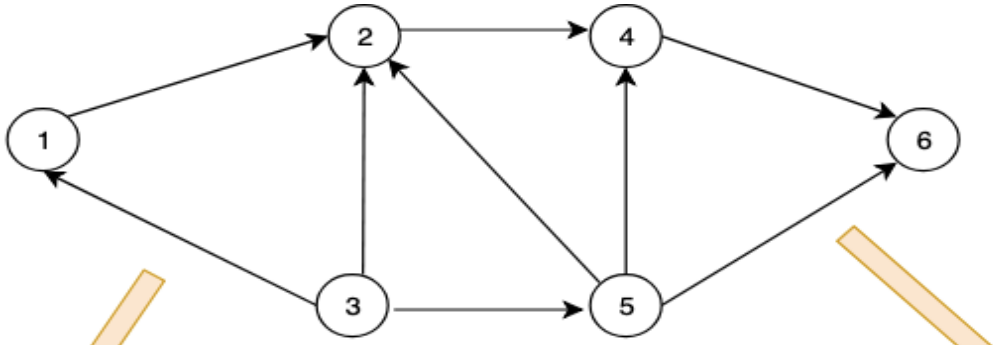


$$\begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{array} \begin{array}{c} \begin{array}{cccccc} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \end{array} \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

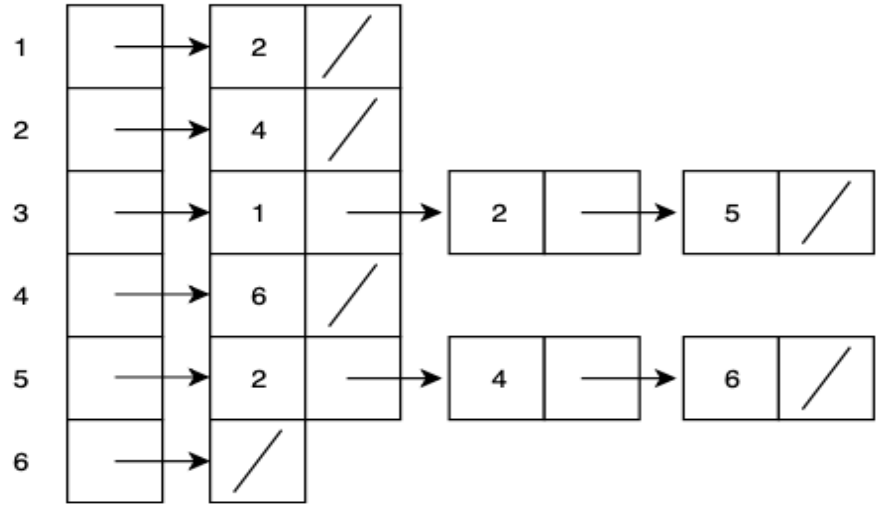


$$\begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{array} \begin{array}{c} \begin{array}{cccccccc} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 & e_8 \end{array} \\ \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \end{array}$$

Directed Adjacency lists and Matrices



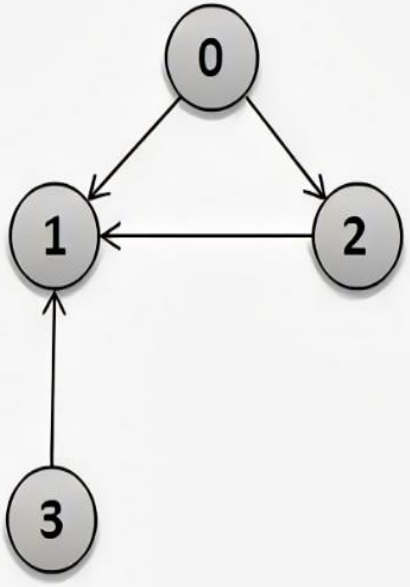
Adjacency List



Adjacency Matrix

	1	2	3	4	5	6
1	0	1	0	0	0	0
2	0	0	0	1	0	0
3	1	1	0	0	1	0
4	0	0	0	0	0	1
5	0	1	0	1	0	1
6	0	0	0	0	0	0

Example of Adjacency Matrices



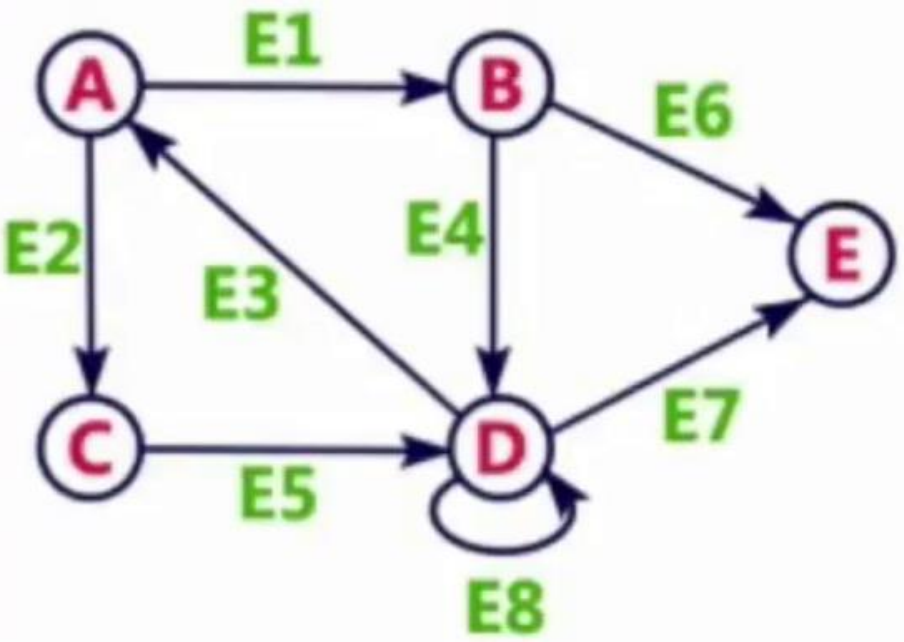
	0	1	2	3
0	0	1	1	0
1	0	0	0	0
2	0	1	0	0
3	0	1	0	0

Incidence matrix is that matrix which represents the graph such that with the help of that matrix we can draw a graph. This matrix can be denoted as $[A_C]$ As in every matrix, there are also rows and columns in **incidence matrix** $[A_C]$.

The rows of the matrix $[A_C]$ represent the number of nodes and the column of the matrix $[A_C]$ represent the number of branches in the given graph. If there are 'n' number of rows in a given incidence matrix, that means in a graph there are 'n' number of nodes. Similarly, if there are 'm' number of columns in that given incidence matrix, that means in that graph there are 'm' number of branches.

The incidence matrix can be applied only to directed graph only. The number of entries in a row apart from zero tells us the number of branches linked to that node. This is also called as degree of that node. The rank of complete incidence matrix is (n-1), where n is the number of nodes of the graph.

Incident Matrix Example



$$
\begin{array}{c|cccccccc}
 & E1 & E2 & E3 & E4 & E5 & E6 & E7 & E8 \\
\hline
A & 1 & 1 & -1 & 0 & 0 & 0 & 0 & 0 \\
B & -1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\
C & 0 & -1 & 0 & 0 & 1 & 0 & 0 & 0 \\
D & 0 & 0 & 1 & -1 & -1 & 0 & 1 & 1 \\
E & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 0
\end{array}
$$

Introduction to Graph Traversal

Definition: Visiting all the nodes in a graph in a systematic way.

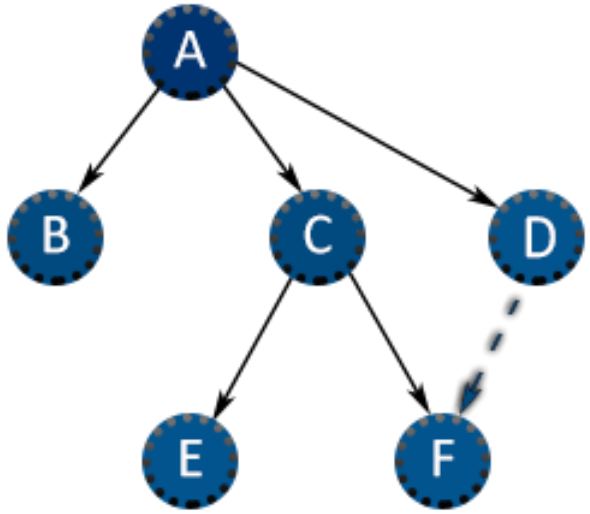
- **Applications**:
- Pathfinding (GPS)
- Web crawling
- Social network analysis
- Network broadcasting

Types of Graph Traversal

- **Breadth-First Search (BFS)**
- **Depth-First Search (DFS)**

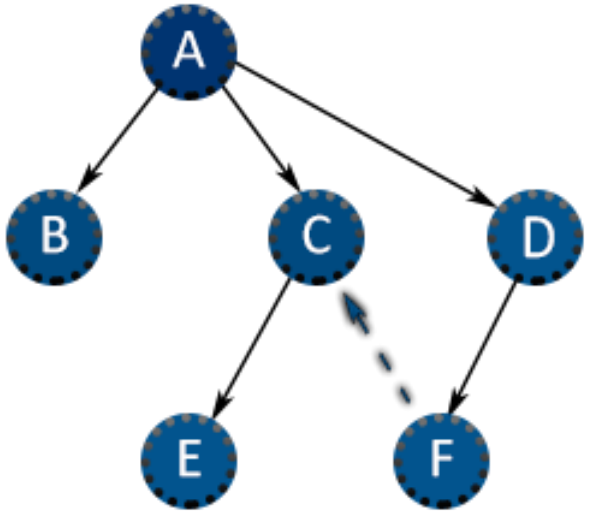
Introduction to Graph Traversal

BFS



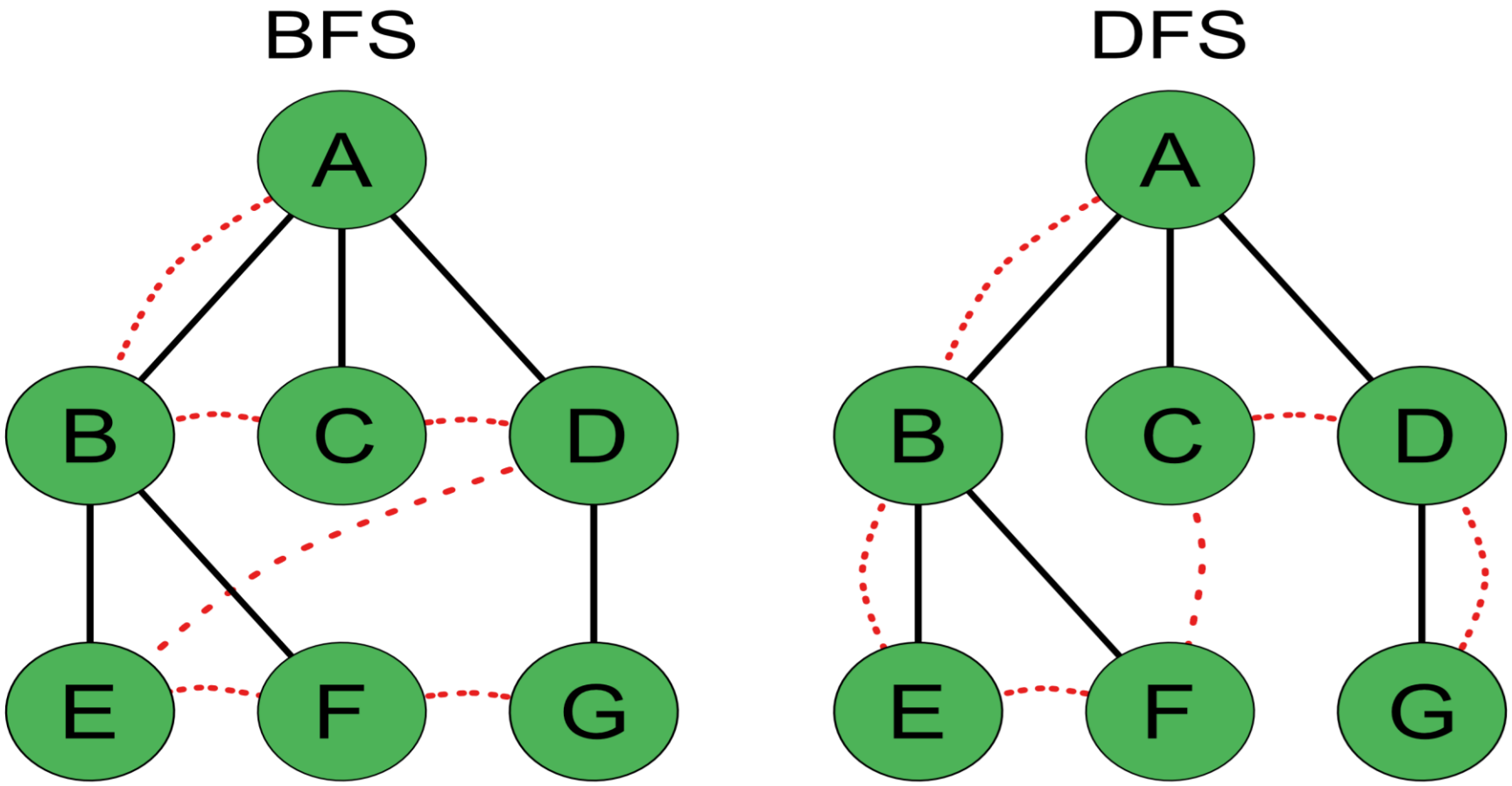
A B C D E F

DFS



A D F C E B

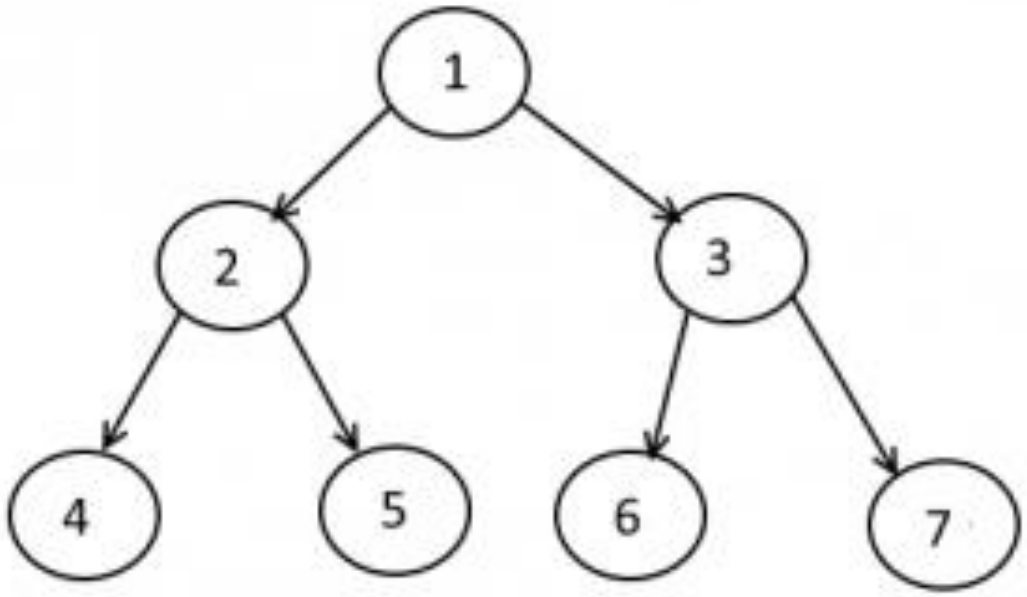
Introduction to Graph Traversal



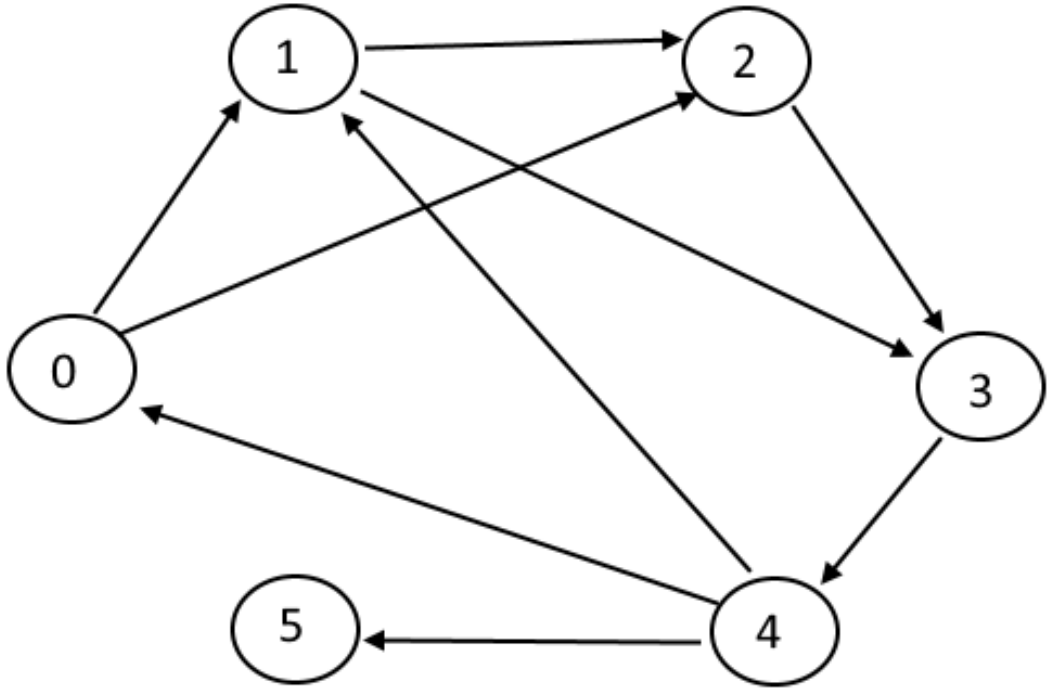
Breadth-First Search (BFS)

- Explores **neighbors** level-by-level.
- Uses a **queue** (FIFO).
- Works well for **shortest path** in unweighted graphs.

Breadth-First Search (BFS)



BFS Traversal - 1 2 3 4 5 6 7



BFS starting from Node 0

0 2 1 3 4 5

Breadth-First Search (BFS)

Follow the below given approach:

1.Initialization: Enqueue the given source vertex into a queue and mark it as visited.

2.Exploration: While the queue is not empty:

2. Dequeue a node from the queue and visit it (e.g., print its value).

3. For each unvisited neighbor of the dequeued node:

4. Enqueue the neighbor into the queue.

5. Mark the neighbor as visited.

Breadth-First Search (BFS)

Breadth-First Search (BFS) Algorithm

Step 1: Define a queue with the same number of nodes as the graph.

Step 2: Start traversal from starting point and put it into the Queue and count it as visited.

Step 3: Enqueue all the neighboring nodes of the visited node that have not been visited yet.

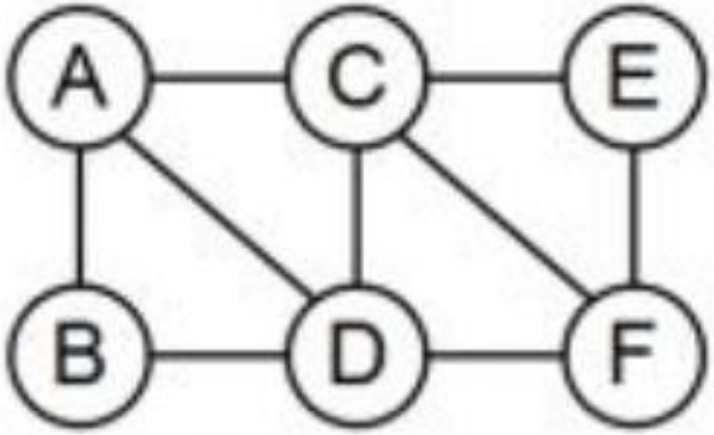
Step 4: When there are no more vertices to visit . Dequeue that node (front Node) from the queue.

Step 5: Repeat steps 3 and 4 until queue is empty.

Step 6: When the queue is empty, make the final spanning tree by removing unused lines from the graph.

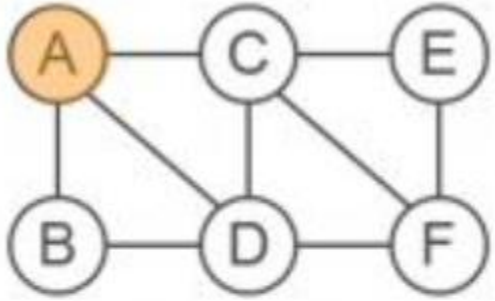
Breadth-First Search (BFS)

Consider the following Graph to perform Breadth First Traversal (BFS)



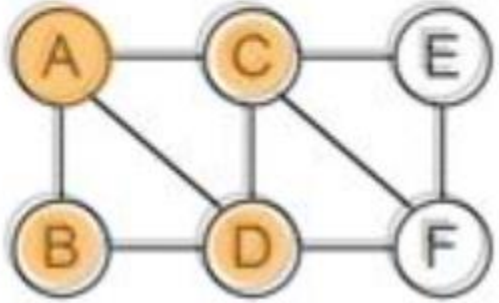
Breadth-First Search (BFS)

STEP 1 - Start with vertix A and add it into Queue



A					

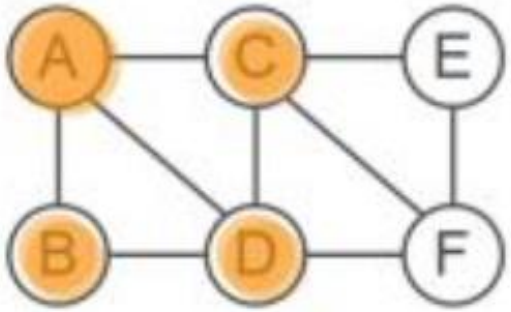
STEP 2 - Visit all vertices attached to A and add them to Queue
Delete A from Queue



	B	C	D		

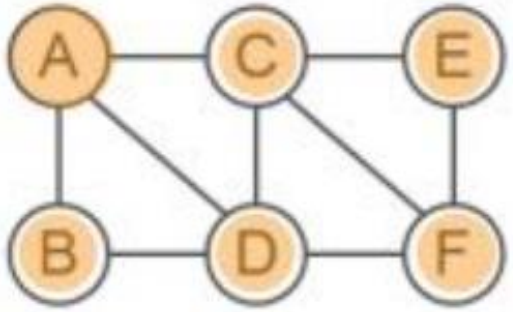
Breadth-First Search (BFS)

STEP 3 - Visit all vertices attached to B (No Vertex)
Delete B from Queue



		C	D		

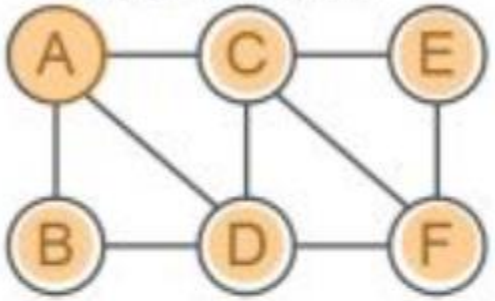
STEP 4- Visit all vertices attached to C which are not visited (E and F) add them to Queue
Delete C from Queue



			D	E	F

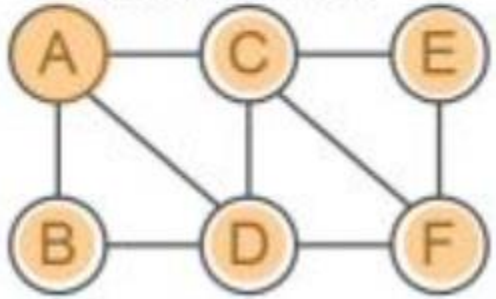
Breadth-First Search (BFS)

STEP 5 - Visit all vertices attached to D which are not visited
Delete D from Queue



				E	F

STEP 6 - Visit all vertices attached to E which are not visited
Delete E from Queue

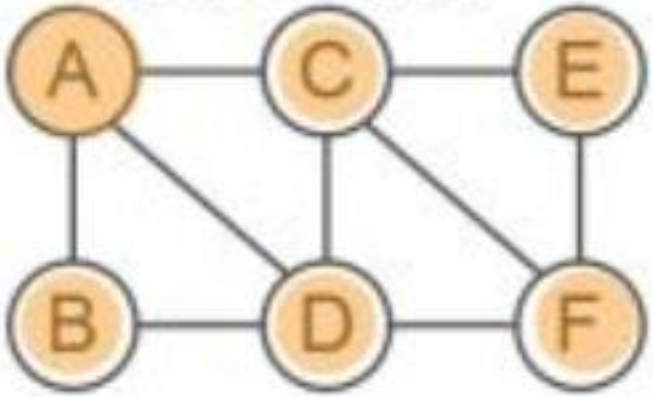


					F

Breadth-First Search (BFS)

STEP 7 - Visit all vertices attached to F which are not visited

Delete F from Queue



Queue is Empty Now, Stop BFS

Depth-First Search (DFS)

- Explores as **deep** as possible before backtracking.
- Uses a **stack** (can be recursion or explicit).
- Useful for topological sort, detecting cycles

Depth-First Search (DFS)

Depth-First Search (DFS) Algorithm

Step 1 - Define a Stack of size equal to the numbers of vertices in the graph..

Step 2 - Choose any vertex as **starting point** for traversal. Visit that vertex and push it on to the Stack.

Step 3 - Push any non-visited adjacent vertex of a top vertex into the stack.

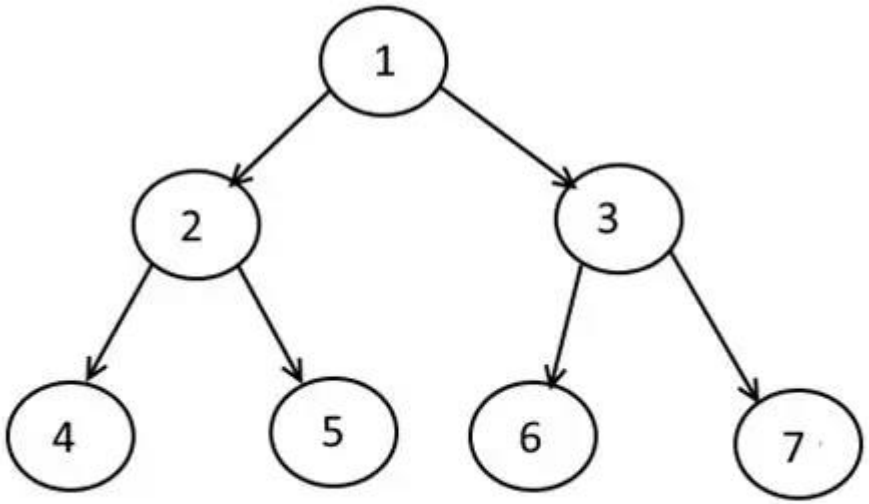
Step 4 - Repeat step 3 until there is no new vertex to visit from the top of the stack.

Step 5 - Backtrack and remove one vertex from the stack if there are no new vertices.

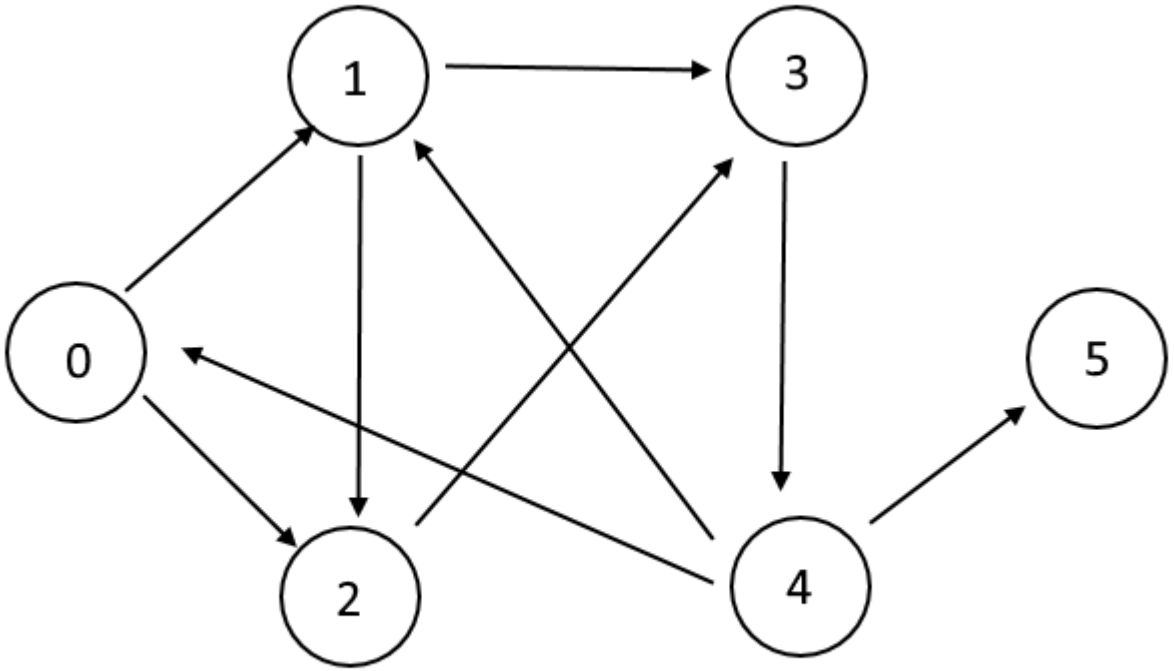
Step 6 - Repeat steps 3, 4 and 5 until stack becomes Empty.

Step 7 - When stack is empty, remove unused graph edges to create final spanning tree.

Depth-First Search (DFS)



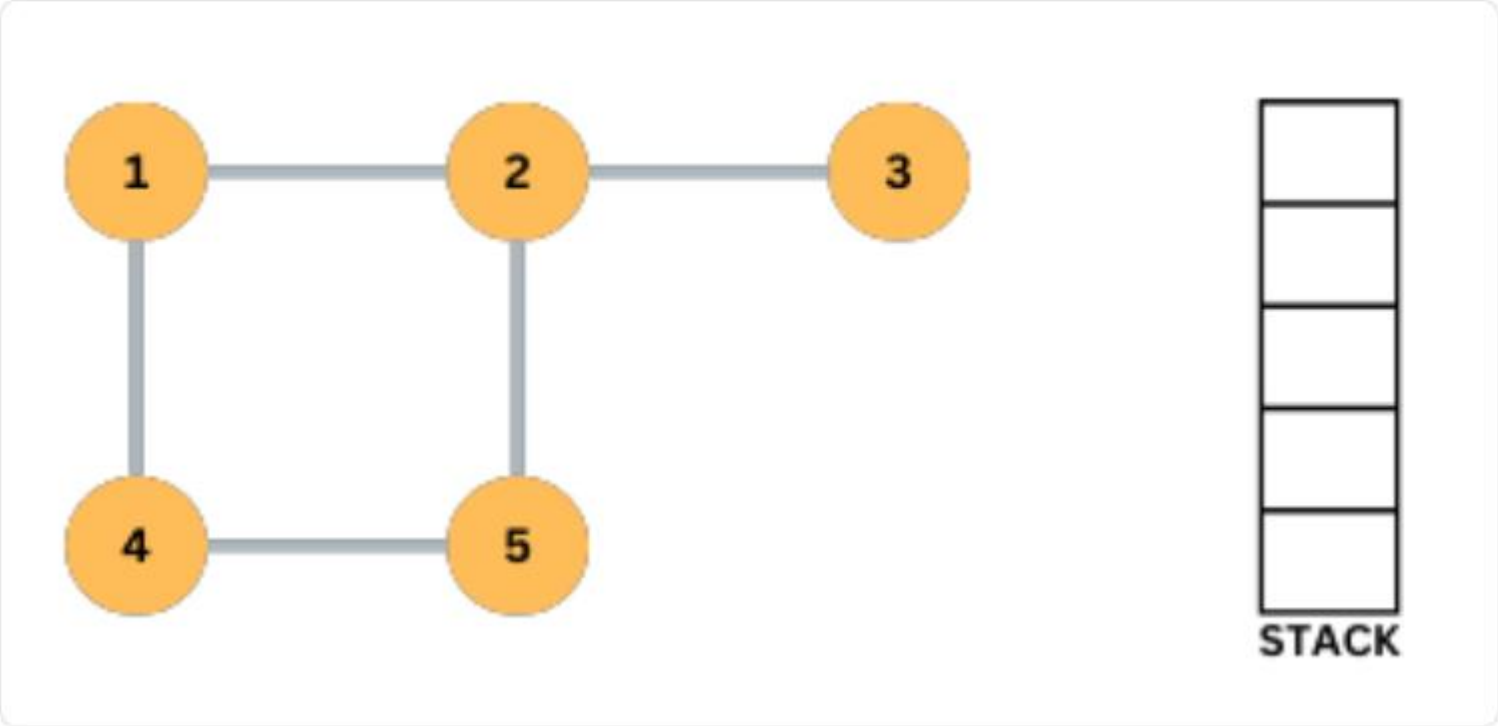
DFS Traversal - 1 2 4 5 3 6 7



Depth First Traversal - 0 1 3 4 5 2

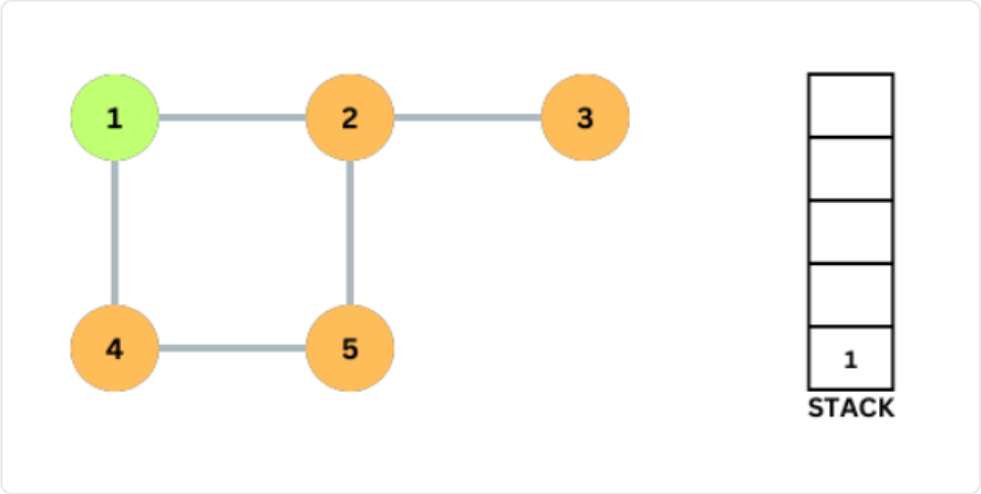
Depth-First Search (DFS)

Consider the following graph for example of DFS

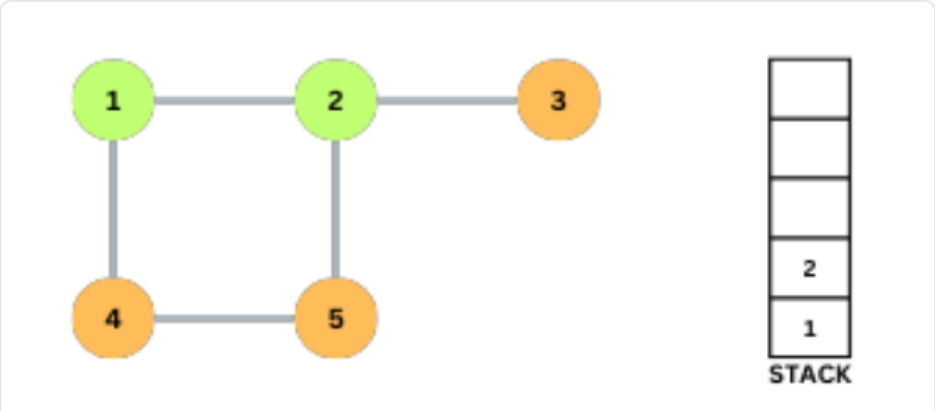


Depth-First Search (DFS)

Step 1: Select the vertex 1 as a starting point (Visit 1). Push 1 into the Stack

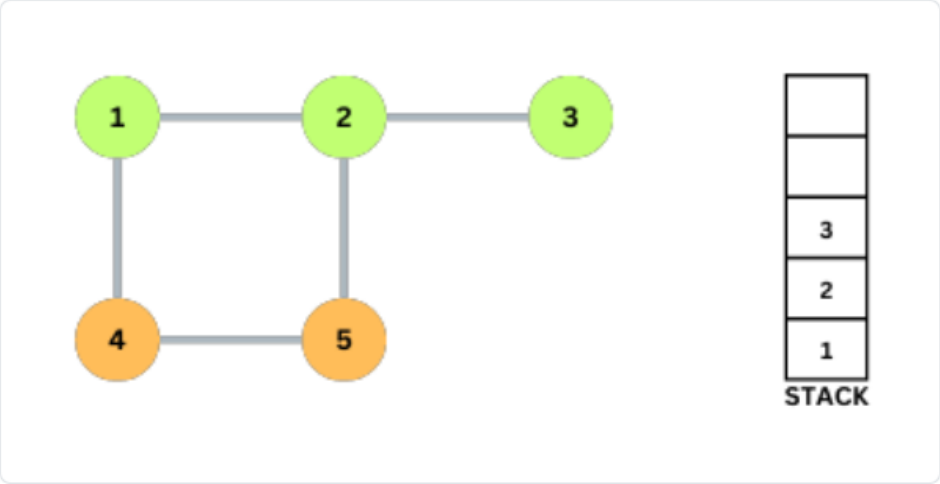


Step 2: Visit any non-visited adjacent vertex of 1. Push 2 in the stack

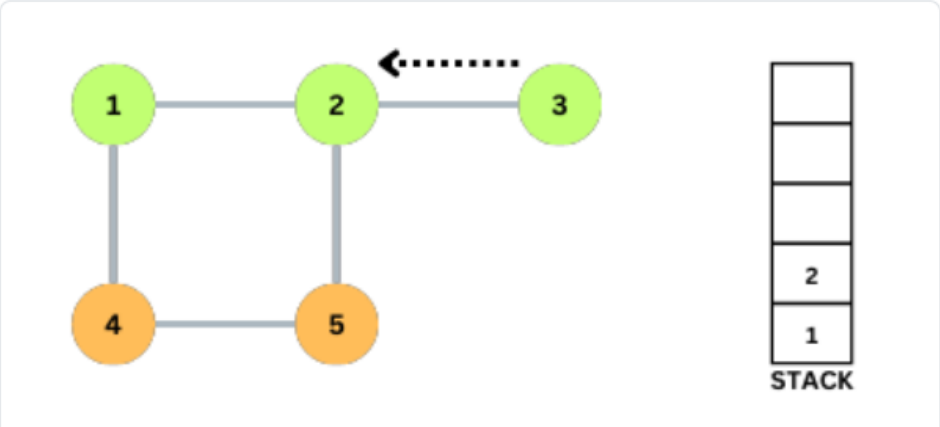


Depth-First Search (DFS)

Step 3: Visit any non-visited adjacent vertex of 2. Push 3 in the stack

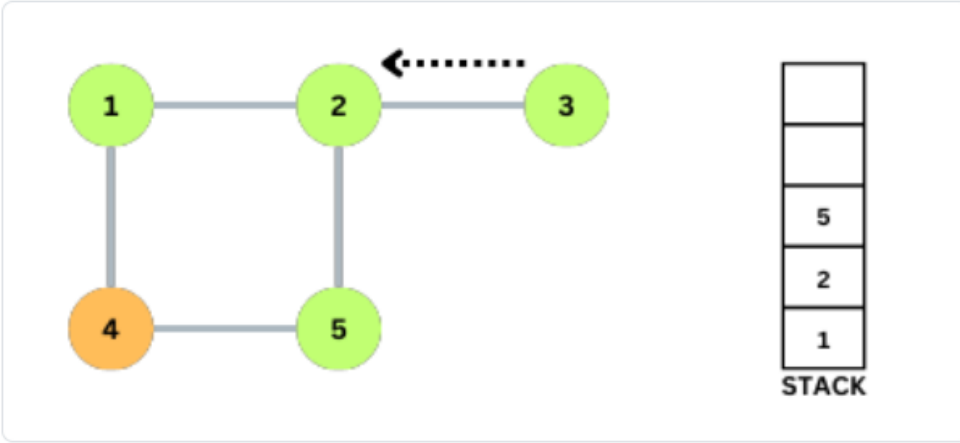


Step 4: There is no vertex to visit from 3. Now Backtrack from 3 and POP 3 from Stack

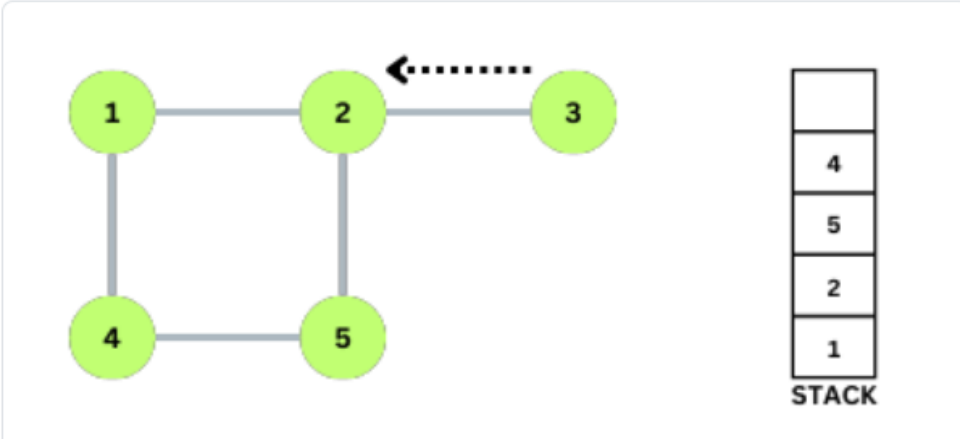


Depth-First Search (DFS)

Step 5: Visit any non-visited adjacent vertex of 2. Push 5 in the stack.

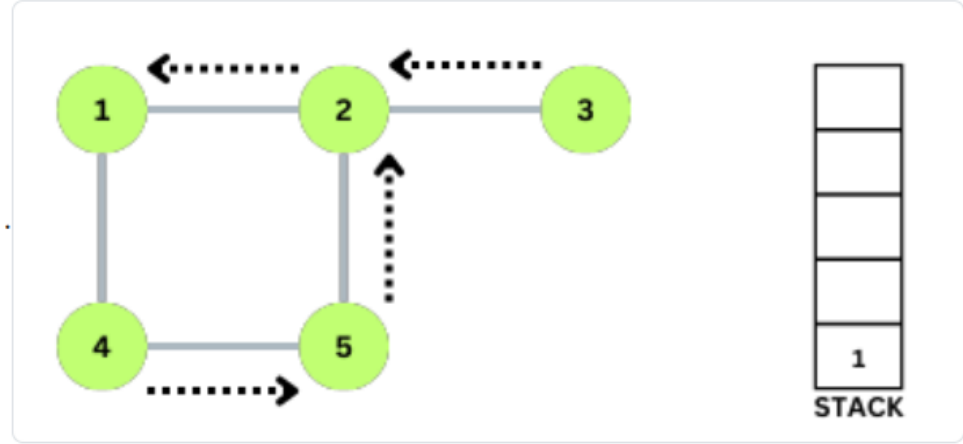


Step 6: Visit any non-visited adjacent vertex of 5. Push 4 in the stack.

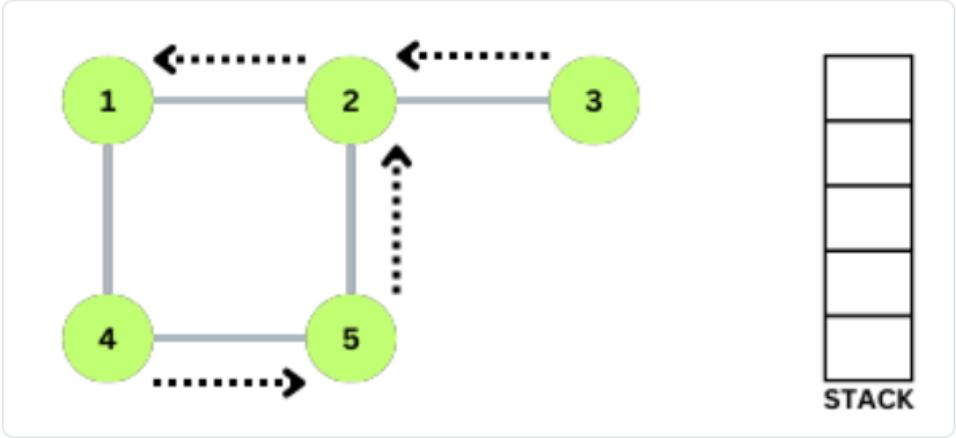


Depth-First Search (DFS)

Step 9: There is no unvisited vertex from 2. Now start backtracking and POP 2 from stack.

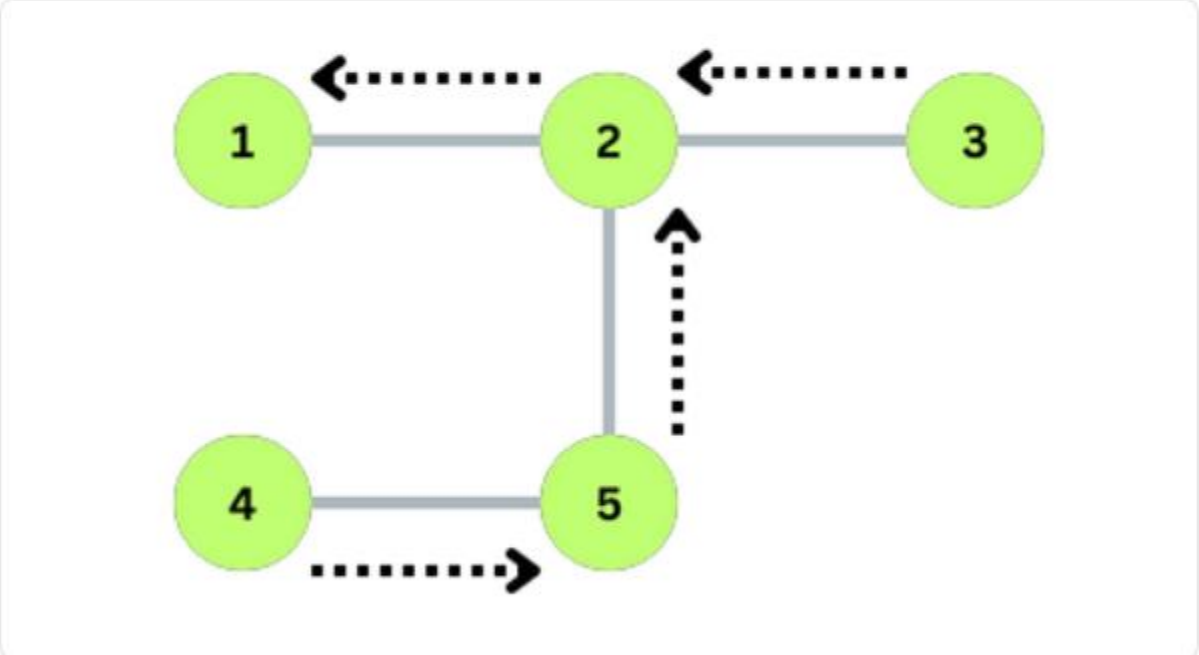


Step 10: There is no unvisited vertex from 1. POP 1 from stack.



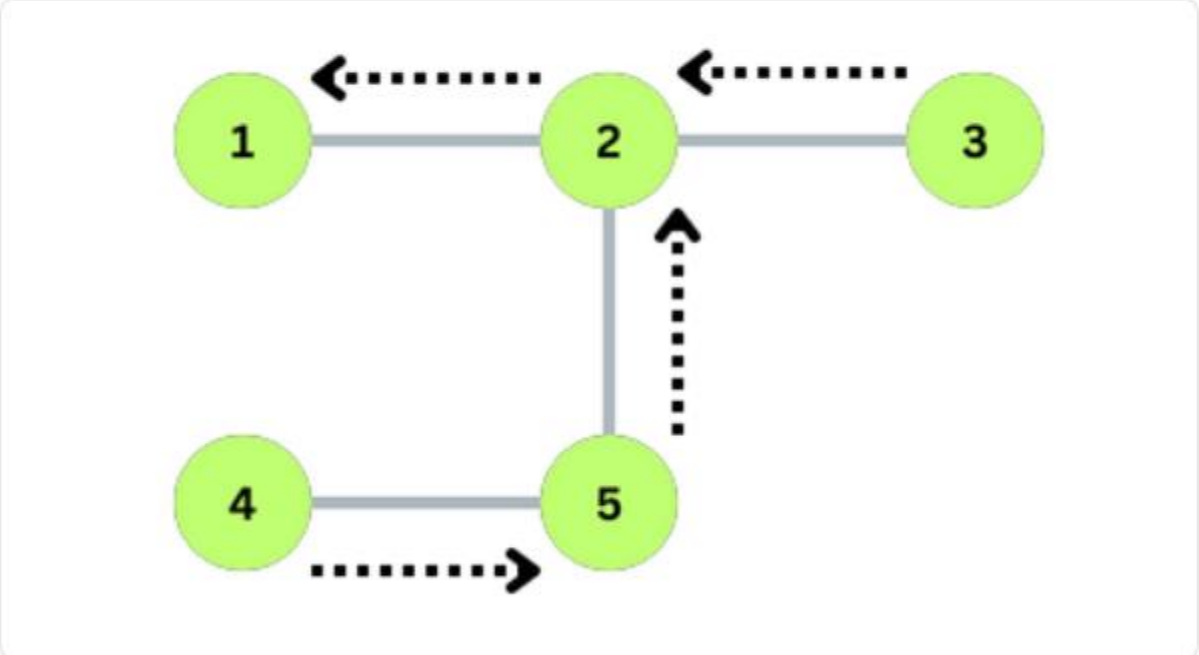
Depth-First Search (DFS)

Step 11: Stack is empty now. Stop DFS Traversal. Final Spanning tree after DFS traversal is given below



Depth-First Search (DFS)

Step 11: Stack is empty now. Stop DFS Traversal. Final Spanning tree after DFS traversal is given below



BFS vs DFS

S.NO	BFS	DFS
1.	BFS stands for Breadth First Search.	DFS stands for Depth First Search.
2.	BFS(Breadth First Search) uses Queue data structure for finding the shortest path.	DFS(Depth First Search) uses Stack data structure.
3.	BFS can be used to find single source shortest path in an unweighted graph, because in BFS, we reach a vertex with minimum number of edges from a source vertex.	In DFS, we might traverse through more edges to reach a destination vertex from a source.
3.	BFS is more suitable for searching verteces which are closer to the given source.	DFS is more suitable when there are solutions away from source.
4.	BFS considers all neighbors first and therefore not suitable for decision making trees used in games or puzzles.	DFS is more suitable for game or puzzle problems. We make a decision, then explore all paths through this decision. And if this decision leads to win situation, we stop.
5.	The Time complexity of BFS is O(V + E), where V stands for vertices and E stands for edges.	The Time complexity of DFS is also O(V + E), where V stands for vertices and E stands for edges.

To be contd…